

Investigación: Calculadora de subredes IPv4 en GNU C con Docker, Kubernetes y Helm

Curso: IC7602 Redes, Tecnológico de Costa Rica, II Semestre 2026

Tarea: Tarea Corta 2

Este documento fue escrito por el estudiante Kevin Alonso Espinoza Barrantes con ayuda para la investigación por el agente de IA: Claude Sonnet 5

La idea es que todo el grupo tenga el mismo modelo mental antes de repartir el trabajo. Hay dos cosas que conviene revisar en el enunciado. La primera es el documento *Política para el Uso de Inteligencia Artificial Generativa.pdf*, porque define qué uso de IA está permitido y cómo se declara. La segunda es la evaluación presencial, que vale el 50% y la hacen todos juntos: cada integrante debe poder explicar cualquier parte del código, no solo la que le tocó.

1. Arquitectura general

El sistema tiene una sola pieza de software, un servidor TCP escrito en C que escucha en el puerto 9666. El cliente es telnet, así que no hay que programarlo. El despliegue va en tres capas: el binario se empaqueta en una imagen Docker, la imagen corre como Pod en Kubernetes y Helm es la herramienta que instala todo con un solo comando.

```
telnet —TCP:9666—> Service (K8s) —> Pod —> contenedor Docker —> ./subnet-server
```

Dentro del código, conviene separar cuatro capas. Así varias personas pueden trabajar en paralelo, y a la IA se le puede pedir cada módulo por separado con una interfaz clara:

Módulo	Responsabilidad
server.c	Sockets, aceptar clientes, hilos, leer y escribir líneas
parser.c	Convertir el texto del comando en una estructura y llamar a la función correcta
ipcalc.c	Cálculos puros con bits: red, broadcast, máscara, clase, subredes aleatorias
routes.c	Tabla de rutas compartida: SET, DEL, ROUTE

La regla clave es que ipcalc.c no sepa nada de sockets ni de texto. Recibe y devuelve números uint32_t. Eso lo hace fácil de probar con pruebas unitarias, que el enunciado pide como buena práctica.

2. El servidor TCP

Ciclo de vida del socket

El patrón clásico de un servidor TCP en C es:

1. `socket(AF_INET, SOCK_STREAM, 0)` crea el socket.
2. `setsockopt(..., SO_REUSEADDR, ...)` evita el error "Address already in use" al reiniciar.
3. `bind()` a `INADDR_ANY` (0.0.0.0) en el puerto 9666.
4. `listen()`.
5. En un ciclo infinito, `accept()` y luego se atiende al cliente.

Hay que usar **0.0.0.0** y no **127.0.0.1**. Dentro de un contenedor, escuchar solo en localhost hace que el servidor sea inaccesible desde fuera del Pod. Es el error más común al dockerizar.

Concurrencia: hilos, no fork()

Para atender varios clientes a la vez hay dos opciones típicas: `fork()` (un proceso por cliente) o `pthread_create()` (un hilo por cliente). **Usen hilos**. La tabla de rutas debe ser compartida: si un cliente hace `SET ROUTE` y otro hace `ROUTE IP`, el segundo tiene que ver la ruta. Con `fork()` cada proceso tendría su propia copia de la memoria y las rutas no se compartirían. Con hilos, la tabla vive en memoria común y se protege con un `pthread_mutex_t`.

```
while (1) {
    int client_fd = accept(server_fd, NULL, NULL);
    pthread_t tid;
    int *arg = malloc(sizeof(int));
    *arg = client_fd;
    pthread_create(&tid, NULL, handle_client, arg);
    pthread_detach(tid);
}
```

Leer líneas que vienen de telnet

TCP es un flujo de bytes y no garantiza que un `recv()` traiga exactamente un comando. El hilo del cliente debe acumular bytes en un buffer hasta encontrar `\n` y luego procesar esa línea. Conviene tener en cuenta tres detalles de telnet:

- Telnet termina las líneas con `\r\n`, así que hay que eliminar el `\r` antes de parsear.
- Telnet normalmente no negocia opciones al conectarse a puertos distintos del 23, pero por seguridad conviene ignorar secuencias que empiecen con el byte `0xFF` (IAC).
- Hay que limitar el tamaño de la línea (por ejemplo 1024 bytes) y responder con error si se excede.

El ciclo del hilo queda así: leer una línea, parsearla, ejecutarla, enviar la respuesta con `\r\n` y repetir hasta que el cliente cierre la conexión (`recv` devuelve 0) o envíe `EXIT/QUIT`. Pueden agregar esos dos comandos y un `HELP` como extra y documentarlos.

Formato de respuesta (el "protocolo")

El enunciado pide implementar un protocolo, así que conviene definirlo y documentarlo. Una propuesta simple:

- Éxito: el resultado tal cual, como en los ejemplos (10.8.2.7).
- Error: una línea que empiece con `ERROR:` y un mensaje útil, por ejemplo `ERROR: máscara inválida '255.0.255.0'` (los bits no son contiguos).

3. El parser de comandos

Se tokeniza la línea con `strtok_r` (la versión segura para hilos) separando por espacios, y se comparan palabras con `strcasemp` para aceptar mayúsculas o minúsculas. Después se decide qué comando es mirando los primeros tokens:

Tokens iniciales	Comando
GET BROADCAST IP	broadcast
GET NETWORK NUMBER IP	número de red
GET HOSTS RANGE IP	rango de hosts
GET HOSTS COUNT MASK	cantidad de hosts
GET NETMASK MASK	conversión de máscara
GET IP CLASS	clase
GET RANDOM SUBNETS NETWORK NUMBER	subredes aleatorias
SET ROUTE / DEL ROUTE / ROUTE IP	rutas

Hay que tener cuidado con que `GET HOSTS RANGE` y `GET HOSTS COUNT` comparten prefijo, igual que las palabras `NETWORK NUMBER` aparecen en dos comandos distintos. Por eso conviene comparar la secuencia completa de palabras clave, no solo la primera.

Cada comando debe validar la cantidad de tokens y la posición de las palabras clave (`IP`, `MASK`, `NUMBER`, `SIZE`, `PRIORITY`, `DEBUG`). Si algo falta, se responde con un error que diga qué se esperaba, por ejemplo `ERROR: uso: GET BROADCAST IP <ip> MASK </n | x.x.x.x>`.

Validar direcciones IP

Se puede usar `inet_pton(AF_INET, str, &addr)` y luego `ntohl()` para trabajar en orden de host, o parsear a mano con `sscanf("%u.%u.%u.%u")` verificando que cada octeto esté entre 0 y 255 y que no sobren caracteres. Internamente, **toda IP y toda máscara se guardan como un `uint32_t`**. Eso hace que todos los cálculos sean operaciones bitwise directas.

Validar máscaras (dos formatos)

Formato /n: `n` debe estar entre 0 y 32. La conversión es:

```
uint32_t mask = (n == 0) ? 0 : (0xFFFFFFFFu << (32 - n));
```

El caso `n == 0` va aparte porque desplazar un entero de 32 bits en 32 posiciones es comportamiento indefinido en C.

Formato X.X.X.X: además de ser una IP válida, los unos deben ser contiguos (255.0.255.0 no es una máscara). El truco con bits es:

```
uint32_t inv = ~mask;
int valida = (inv & (inv + 1)) == 0; // inv debe ser de la forma 000...0111...1
int prefijo = __builtin_popcount(mask); // extensión de GNU C: cuenta los bits en 1
```

__builtin_popcount viene bien porque el enunciado pide GNU C específicamente.

Una buena decisión de diseño es tener una sola función parse_mask(const char *s, uint32_t *mask, int *prefix) que acepte ambos formatos y que todos los comandos la reutilicen.

4. La lógica de cada primitiva (operaciones bitwise)

Con ip y mask como uint32_t:

Primitiva	Cálculo
Número de red	net = ip & mask
Broadcast	bcast = net ~mask
Primer host	net + 1
Último host	bcast - 1
Cantidad de hosts	(1u << (32 - n)) - 2 (con cuidado en n=0 usando uint64_t)
Máscara ↔ prefijo	/n → máscara con el desplazamiento; máscara → /n con popcount

Verificación con el ejemplo del enunciado, 10.8.2.5 /29: la máscara es 255.255.255.248. El último octeto 5 es 00000101; con AND 11111000 da 00000000, así que la red es 10.8.2.0. El broadcast pone los 3 bits de host en 1: 00000111 = 7, es decir 10.8.2.7. Los hosts van de .1 a .6, que son 6. Coincide con el enunciado.

Casos borde que el grupo debe decidir y documentar

- **/31 y /32:** la fórmula da 0 y -1 hosts. El RFC 3021 permite usar /31 en enlaces punto a punto (2 hosts) y /32 representa un solo host. Decidan un comportamiento y pónganlo en la documentación.
- **Formato del rango:** los ejemplos muestran 10.8.2.{1-6}, que solo funciona cuando el rango cambia en el último octeto. Para una /16 (172.16.0.1 a 172.16.255.254) ese formato no alcanza. Pueden usar llaves cuando solo varía el último octeto y el formato 172.16.0.1 - 172.16.255.254 en los demás casos, o preguntarle al profesor. Lo importante es que quede documentado.

Clase de la IP

Se mira el primer octeto, o directamente los bits más altos:

Clase	Primer octeto	Bits iniciales
A	0–127	0

Clase	Primer octeto	Bits iniciales
B	128–191	10
C	192–223	110
D (multicast)	224–239	1110
E (reservada)	240–255	1111

Como detalle extra, pueden indicar que 127.x.x.x es loopback y que 0.x.x.x es especial.

Subredes aleatorias

GET RANDOM SUBNETS NETWORK NUMBER 10.0.0.0 MASK /8 NUMBER 3 SIZE /24

La lógica es:

1. Validar que SIZE sea mayor o igual que MASK (no se puede sacar una /8 de una /24).
2. Validar que la red base esté alineada (`net & mask == net`) o normalizarla.
3. La cantidad de subredes posibles es $total = 2^{(size - mask)}$; en el ejemplo, $2^{16} = 65536$.
4. Validar que `NUMBER <= total`.
5. Elegir NUMBER índices aleatorios **distintos** entre 0 y `total-1`.
6. Cada subred se obtiene como `net | (indice << (32 - size))`.

En el ejemplo, el índice 0x140A da `10.0.0.0 | (0x140A << 8) = 10.20.10.0/24`.

Para no repetir subredes, lo más simple es guardar las ya elegidas y reintentar si sale una repetida, lo cual funciona bien cuando NUMBER es pequeño respecto a total. Como el servidor usa hilos, usen `rand_r()` con una semilla por hilo o lean de `/dev/urandom`, porque `rand()` no es seguro entre hilos.

5. Tabla de rutas y routing

Cada ruta es una estructura:

```
typedef struct {
    char name[64];
    uint32_t network;
    uint32_t mask;
    int prefix;
    int priority;
} route_t;
```

La tabla puede ser un arreglo o una lista enlazada global, protegida con un `pthread_mutex_t` en cada lectura y escritura.

- **SET ROUTE nombre NETWORK NUMBER Y.Y.Y.Y MASK m PRIORITY p:** se valida la IP, la máscara y que la red esté alineada con la máscara. También hay que decidir qué pasa si el nombre ya existe: error o sobrescribir.
- **DEL ROUTE nombre:** se elimina, o se responde con error si no existe.

- **ROUTE IP x.x.x.x DEBUG true/false:** es la parte más interesante y la que probablemente pregunten en la evaluación presencial.

Algoritmo de routing

Es el mismo que usan los routers reales, **longest prefix match**:

1. Para cada ruta, se verifica si coincide: `(ip & route.mask) == route.network`.
2. Entre las que coinciden, gana la de **prefijo más largo** (la más específica).
3. Si hay empate de prefijo, se desempata con **PRIORITY**. El grupo decide si menor número significa mayor prioridad, como en las métricas de routing, o al revés, y lo documenta.
4. Si ninguna coincide, se responde **ERROR**: no hay ruta para 8.8.8.8, salvo que exista una ruta por defecto `0.0.0.0/0`, que coincide con cualquier IP.

Modo DEBUG

Con `DEBUG true`, el servidor muestra el razonamiento. Por ejemplo:

```
Evaluando 192.168.1.50 (11000000.10101000.00000001.00110010)
ruta LAN 192.168.1.0/24 -> 192.168.1.50 & 255.255.255.0 = 192.168.1.0 COINCIDE (prefijo
24, prioridad 1)
ruta WAN 0.0.0.0/0 -> 192.168.1.50 & 0.0.0.0 = 0.0.0.0 COINCIDE
(prefijo 0, prioridad 5)
ruta VPN 10.0.0.0/8 -> 192.168.1.50 & 255.0.0.0 = 192.0.0.0 NO coincide
Seleccionada: LAN (prefijo más largo)
LAN
```

Mostrar las IPs en binario luce muy bien y demuestra dominio de los operadores bitwise, que es uno de los objetivos específicos.

6. Docker: empaquetar el servidor

Se recomienda un **Dockerfile multi-stage**. La primera etapa compila con `gcc` y la segunda solo copia el binario a una imagen pequeña:

```
# Etapa 1: compilación
FROM gcc:14 AS build
WORKDIR /app
COPY . .
RUN make

# Etapa 2: ejecución
FROM debian:bookworm-slim
WORKDIR /app
COPY --from=build /app/subnet-server .
EXPOSE 9666
CMD ["/subnet-server"]
```

Para que el servidor se comporte bien dentro de un contenedor:

- Escuchar en `0.0.0.0`, como se mencionó antes.
- Escribir los logs en `stdout/stderr`, así se ven con `docker logs` y `kubectl logs`. Usen `setvbuf(stdout, NULL, _IONBF, 0)` o `fflush` para que no se queden en el buffer.

- Manejar SIGTERM para cerrar ordenadamente, porque Kubernetes lo envía al detener un Pod.
- Ignorar SIGPIPE con `signal(SIGPIPE, SIG_IGN)`, para que el servidor no muera si un cliente cierra telnet mientras se le está escribiendo.

Prueba local rápida antes de pasar a Kubernetes:

```
docker build -t subnet-calculator:1.0 .
docker run -p 9666:9666 subnet-calculator:1.0
telnet localhost 9666
```

7. ¿Qué es Kubernetes y qué es Helm?

Kubernetes es un orquestador de contenedores. Uno le describe en archivos YAML el estado deseado ("quiero 1 réplica de esta imagen, expuesta en este puerto") y Kubernetes se encarga de lograrlo y mantenerlo: crea los Pods, los reinicia si mueren, etc. Para esta tarea bastan dos recursos:

- **Deployment:** define qué imagen correr, cuántas réplicas, puertos y probes de salud.
- **Service:** da una dirección estable para llegar a los Pods. Para conectarse con telnet desde fuera del clúster se usa tipo NodePort (abre un puerto en el nodo, en el rango 30000–32767) o `kubectl port-forward`.

Helm es el gestor de paquetes de Kubernetes, algo así como `apt` o `npm` pero para aplicaciones de Kubernetes. En vez de aplicar a mano varios YAML con `kubectl apply`, se crea un **chart**: un paquete de plantillas YAML con variables. Los valores configurables (imagen, puerto, tipo de Service) viven en `values.yaml`, y Helm los inyecta en las plantillas usando la sintaxis de Go templates (`{{ .Values.algo }}`). Cada instalación se llama **release**, y Helm permite actualizarla (`helm upgrade`), revertirla (`helm rollback`) y eliminarla (`helm uninstall`) con un comando. Por eso el enunciado lo pide: automatiza la instalación.

Estructura del chart

```
helm/subnet-calculator/
├── Chart.yaml          # nombre, versión y descripción del chart
├── values.yaml         # valores configurables
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── _helpers.tpl    # funciones reutilizables (nombres, labels)
│   └── NOTES.txt       # mensaje que aparece tras instalar (ej. cómo conectarse)
```

Ejemplos

`values.yaml`:

```
image:
  repository: subnet-calculator
  tag: "1.0"
  pullPolicy: IfNotPresent
replicaCount: 1
service:
  type: NodePort
  port: 9666
  nodePort: 30966
```

templates/deployment.yaml (fragmento):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
        - name: server
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 9666
          readinessProbe:
            tcpSocket:
              port: 9666
          livenessProbe:
            tcpSocket:
              port: 9666
```

Un punto importante para la evaluación: **mantengan replicaCount: 1**. La tabla de rutas vive en memoria del proceso. Con 2 réplicas, el Service repartiría las conexiones entre Pods distintos, y un SET ROUTE hecho en un Pod no existiría en el otro. Esto es una buena conclusión o recomendación para la documentación, y una buena respuesta si les preguntan por qué no escalaron.

La imagen dentro del clúster

Si usan **minikube**, el clúster no ve las imágenes del Docker local. Hay tres opciones:

- `minikube image load subnet-calculator:1.0` después de construirla.
- `eval $(minikube docker-env)` antes del `docker build`, para construir directamente dentro de minikube.
- Subirla a Docker Hub o GitHub Container Registry y apuntar `values.yaml` ahí.

8. Automatización

El enunciado es estricto con esto. Hagan un script `install.sh` (o targets en el Makefile) que haga todo desde cero:

```
#!/bin/bash
set -e
minikube status >/dev/null 2>&1 || minikube start
docker build -t subnet-calculator:1.0 .
minikube image load subnet-calculator:1.0
helm upgrade --install subnet-calc ./helm/subnet-calculator
echo "Conéctese con: telnet $(minikube ip) 30966"
```

`helm upgrade --install` sirve tanto para la primera instalación como para las siguientes. Agreguen también un `uninstall.sh` con `helm uninstall subnet-calc`.

9. Estructura de repositorio sugerida

```
├── src/
│   ├── main.c
│   ├── server.c / server.h
│   ├── parser.c / parser.h
│   ├── ipcalc.c / ipcalc.h
│   └── routes.c / routes.h
├── tests/
│   └── test_ipcalc.c
├── helm/subnet-calculator/
├── docs/                # Markdown → PDF, diagramas
├── Dockerfile
├── Makefile              # make, make test, make docker, make deploy
├── install.sh
└── README.md
```

Para las pruebas unitarias alcanza con `assert()` o con un framework ligero como Unity o CMocka. Como `ipcalc.c` son funciones puras, se prueban sin levantar el servidor. Para las pruebas de integración se pueden automatizar comandos con netcat, por ejemplo `printf "GET IP CLASS 10.8.2.5\r\n" | nc localhost 9666`, y documentar los resultados esperados.

10. Cómo repartir el trabajo entre el grupo

Una posible división para 5 personas:

Persona	Parte
1	<code>server.c</code> : sockets, hilos, lectura por líneas, señales
2	<code>parser.c</code> : tokenización, despacho de comandos, mensajes de error
3	<code>ipcalc.c</code> y sus pruebas unitarias (primitivas 1 a 6)
4	Subredes aleatorias, <code>routes.c</code> , longest prefix match y modo DEBUG
5	Dockerfile, chart de Helm, scripts de automatización y la base de la documentación